

Day 6 — Rasa CALM: Architecture & Project Anatomy

Objective: Introducing the Rasa framework and its CALM architecture, explain the core concepts behind, while also framing it against classic NLU pipelines and ReAct-style agents to resolve the same problem, with hands-on setup of a basic CALM agent under the Developer Edition.

§1 - Introduction to Rasa Framework and CALM architecture

Takeaway: Brief introduction to Rasa as a framework, its history presented briefly, the choice of developing CALM continuing the legacy over the NLU past of the framework, areas of application of the framework, and setting the ground for the rest of the lessons focusing on RASA.

- Rasa Introduction
 - Brief history of the framework and how it evolved till CALM
 - What is Rasa for in a nutshell, and what problems it aims to solve
- Product overview: features, capabilities, use cases
- Rasa use cases: Customer Experience & Support, Process Automation, Sales & Marketing
- Rasa features: flow orchestration With AI Agents, integration with existing NLU, real-time voice, MCP Integration and more
- The advantage of using Rasa to bridge on existing, NLU-based, in production systems
- What is CALM in the context of the Rasa Framework
- Our setup for the course
 - What version of Rasa do we use in this course, and what docs to reference

§2 — The CALM architecture: understand, decide, respond

Takeaway: High level overview of the concepts behind the CALM architecture, what conceptually means each stage: Dialogue Understanding, Dialogue Management, Response Generation, what are the capabilities at an intuitive level, what are the benefits of this approach.

- High level overview of the Rasa main concepts: Dialogue Understanding, Dialogue Management, Response Generation
- conceptual introduction of flows and slots
- How a natural language request is handled by a CALM Agent, schematically: Dialogue Understanding → Dialogue Management → Response
- Benefits of this approach: Separation of Concerns, Determinism, Efficiency, Built-in Conversational Awareness
- In depth: Dialogue Understanding
- context aware understanding through an LLM, the entire conversation is visible

- next steps identifications through "commands", and what they are
- most common commands: `start flow`, `set slot`
- other advanced commands: `cancel flow`, `clarify flows`, `set slot`, `correct slot`, `chitchat`, `knowledge answer`, `human handoff`
- simple example: message to commands
- the command generator: can be an LLM or an NLU model
- LLM vs NLU: pros and cons, and the hybrid approach
- structured output as an architectural guarantee: a validated command vocabulary, not free text
- In depth: Dialogue Management
- the business logic layer, receives commands, performs steps
- anatomy of a flow: name, description, steps
- step building block business logic: Collect, Action, Set Slots, Link, Call, Conditions
- conversation patterns overview: the built in system flows modeled over human interaction.
- conversation patterns details: Repair, Navigation, External Support, Voice, System Error
- In depth: Response
- default approach: templated, predictable responses from the domain
- contextual rephraser: going beyond the template with an LLM
- contextual rephraser benefits: more natural, context aware, controllable, customizable
- contextual rephraser drawbacks: privacy, hallucination, prompt injection, latency

§3 — How CALM compares: classic NLU, ReAct agents, and CALM

Takeaway: Comparison of CALM among the three broad ways to build a conversational assistant — classic intent-based NLU, ReAct-style single-LLM agents, and CALM.

- The three approaches, neutrally
 - classic NLU / intent-based: one intent (+ entities) per message from a fixed catalog; hand-written stories/rules/trees decide next; deterministic, cheap, low-latency, traceable — but off-catalog blindness, brittle trees, overlap-driven accuracy loss, training data per phrasing
 - ReAct-style agents: one LLM understands AND decides each turn, logic in the prompt; flexible, open-ended, minimal scaffolding — but inconsistent decisions, hard to debug, high cost/latency, degrades as tasks pile up
 - CALM: LLM understands in full context, deterministic flows decide — "LLMs keep the conversation fluent but don't guess your business logic"
- Where CALM lands + honest trade-offs
 - autonomy spectrum (workflows ↔ agents); CALM toward the workflow end
 - code-first alternative benefits: the better tool for open-ended/one-off work
 - what CALM buys: auditability, determinism, guardrail placement, team legibility, evaluation surface

- what CALM costs: less flexibility, learning curve, YAML ceremony, unchanged LLM cost/latency
- when Rasa is the wrong tool: open-ended research, one-off automation, unmappable task

§4 — Editions, licensing & installation

Takeaway: Basic introduction to the available editions, licensing information and limitations, installation steps.

- Editions, precisely
 - Developer Edition: free, full Rasa Pro + CALM, community support
 - Business/Enterprise: adds Rasa Studio (no-code), scale, support
 - this course: pro-code CLI path, Developer Edition assumptions throughout
- The restrictions, precisely
 - 1,000 conversations/month external (100 internal); one use case per organisation
 - 12-month key; production use allowed within caps
 - no professional financial advice — informational content only
- Installing Rasa Pro 3.17
 - uv + Python 3.11 (3.10 also supported; 3.12/3.13 fine for CALM-only); rasa-pro from public PyPI
 - RASA_LICENSE env var
 - LLM provider key is mandatory: CALM does not run without a model
 - providers at a glance
- The CLI workflow loop
 - rasa init → edit → rasa train → rasa inspect
 - rasa shell and rasa run, named for later

§5 — Project anatomy: the scaffold tour

**Takeaway: Basic information about the available project scaffolds templates, high level conceptual understanding of the main YAML files, understanding on how to map each component of CALM (dialogue understanding, dialogue management, response generation) to the project structure. **

- Scaffolding
- rasa init --template default — the CALM default scaffold
- available templates: default, tutorial, basic, finance, telco
- Overview of the most important files to start customizing a CALM agent
- config.yml: high level overview of what it enables
- config.yml: FlowPolicy in short
- domain.yml: high level overview of what it enables

- domain.yml: slots, responses, actions
- flows.yml: high level overview of what it enables
- flows.yml: anatomy of a flow, steps, and how to read it
- brief overview: endpoints.yml, credentials.yml
- Anchoring to the architecture
 - each file mapped onto the Dialogue Understanding, Dialogue Management, Response Generation stages

§6 — The Rasa Inspector, a brief intro

Takeaway: Knows what `rasa inspect` is for — a browser-based local tool to chat with the last-trained assistant and watch CALM's internals live — and can name its three panels and what each reveals.

- What it is and how to launch
 - `rasa inspect` runs the last-trained model locally, opens in the browser
 - chat-only by default; the **Inspect** button reveals the panels
- The three panels at a glance
 - Preview: chat + inline events (slots set, actions, the command the model emitted); execution trace made legible
 - Flow: live flowchart of the active flow, current step highlighted
 - History & Memory: timeline of flow invocations (status badges) + slots by scope (current-flow / session / system)